Tema 3

Métodos simples de Aprendizaje Automático Supervisado

3.1 Material de estudio

Los materiales de estudio de este tema son: el capítulo 41 In Depth: Naive Bayes Classification de Python Data Science Handbook, 2nd Edition, el capítulo 9 de Python Machine Learning - Third Edition y el capítulo 6 de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition completos y adquirir un conocimiento de las herramientas que en ellos se comenta principalmente a partir de la documentación de scikit-learn. Iremos detallando el uso de estos materiales en las secciones siguientes. Si vais a usar Google Colab deberéis subir los notebooks de los dos primeros libros.

Para repasar conceptos de probabilidad, no hay que conocerlos en profundidad solo tener una idea de ellos, recomendamos: Introduction to Probability o consultar las páginas correspondientes de Wikipedia especialmente en inglés, la versión en español suele ser un resumen.

Los códigos de ejemplo nuevos en este tema son PythonDataScienceHandbook_master.zip y Python_Machine_Learning_Source_Code.zip disponibles directamente en los Documentos de Alf en la carpeta code.

3.2 Índice

- Naive Bayes.
- K-Nearest-Neighbors

- Árboles de decisión

3.3 Naive Bayes

Naive Bayes (Bayes ingenuo) son un conjunto de técnicas de aprendizaje supervisado basadas en la aplicación del teorema de Bayes asumiendo que las características son independientes entre sí dada una clase, o sea, no se tienen en cuenta correlaciones entre variables. En el capítulo 41 In Depth: Naive Bayes Classification de Python Data Science Handbook, 2nd Edition, se describe esta idea, su base matemática y se comentan y dan ejemplos de dos clasificadores de este tipo: Gaussian Naive Bayes, Multinomial Naive Bayes.

En la sección de Gaussian Naive Bayes el notebook 05.05-Naive-Bayes.ipynb de PythonDataScienceHandbook_master contiene el ejemplo y hace uso del generador de muestras aleatorias make_blobs de scikit-learn, estos generadores permiten construir datos artificiales para pruebas controladas.

En la sección de Multinomial Naive Bayes se comenta un ejemplo de clasificación de textos, está en el notebook comentado anteriormente, los textos deben ser convertidos en números para que este y cualquier otro clasificador pueda trabajar con ellos. En el texto hay un enlace a la sección Feature Engineering donde se da una introducción a la vectorización de textos, estos temas se tratarán con mayor profundidad en la asignatura Minería de textos, aquí solo se trata de saber que existen esos procedimientos y su uso simple.

Completaremos el estudio de este tema con la lectura y estudio de la documentación scikit-learn sobre Naive Bayes, en ella se comentan otros dos procedimientos: Complement Naive Bayes y Bernoulli Naive Bayes. La sección Out-of-core naive Bayes model fitting no constituye material de estudio. En el enlace a la documentación de Naive Bayes se incluyen enlaces a las APIs de los 4 procedimientos motivo de estudio que es muy conveniente conocer para su aplicación práctica. Los ejemplos incluidos al final de las secciones de las APIs son de lectura opcional, el ejemplo corto situado antes de estas secciones da una visión rápida de cómo emplear las distintas herramientas.

3.4 K-Nearest-Neighbors (KNN)

Vamos a ver el uso de KNN para tareas de clasificación y regresión. El uso de KNN para aprendizaje no supervisado no es materia de este tema.

3.4.1 Clasificación

Como su nombre indica este algoritmo clasifica una instancia según la clase predominante entre sus K vecinos más cercanos. En el capítulo 9 de Python Machine Learning se comenta este algoritmo. Básicamente el capítulo tiene dos secciones: Implementing Knn in Python y Using scikit-learn'S Kneighborsclassifier Class for Knn. De la primera no es necesario conocer cómo se implementa el código, sino tener la idea de qué está haciendo es código, ya que es la descripción del algoritmo. De la segunda sí hay que adquirir un conocimiento detallado ya que explica en profundidad cómo aplicar KNN y encontrar el valor óptimo de K. Para ello usa un dataset de juguete de scikit-learn dataset de juguete de scikit-learn, load_iris, si antes de la carga del dataset `iris = datasets.load_iris()` se añade esta línea `from IPython.display import Markdown` y tras la carga esta otra `display(Markdown(iris.DESCR))`, se mostrará la descripción del dataset, si se usara directamente `print(iris.DESCR)` la salida sería un tanto desordenada. Al ver la descripción nos daremos cuenta, entre otras cosas, de porqué habla de sépalos en el notebook y cuáles son los tipos de iris.

El código del ejemplo se halla en el notebook Chapter 9 - Supervised Learning—Classification Using K-Nearest Neighbors (KNN).ipynb de Python Machine Learning Source Code. En el se usa el paquete seaborn de visualización estadística de datos que está basado en Matplotlib, no será materia de estudio de la asignatura pero se puede usarlo si se desea, viene instalado por defecto en Anaconda y en Google Colab. Una nota sobre el cálculo del step de numpy.meshgrid, en el notebook se calcula así `h = (x_max / x_min)/100`, si x_min fuera 0 habría un error, creemos que quisieron poner `h = (x_max - x_min)/100`.

Para completar el estudio de KNN para clasiificación consultar de la documentación de scikit-learn Nearest Neighbors la introducción y la sección 1.6.2 y los enlaces a las APIs contenidos en 1.6.2: KNeighborsClassifier y RadiusNeighborsClassifier. El ejmplo al final de la sección 1.6.2 no es necesario verlo ya que clasifica iris de la misma manera que el ejemplo del libro pero con menos profundidad.

3.4.2 Regresión

En el libro omite el estudio de la regresión usando KNN. Para estudiarlo haremos uso de la sección 1.6.3 de la documentación de scikit-learn Nearest Neighbors y los enlaces a las APIs: KNeighborsRegressor y RadiusNeighborsRegressor. Como ejemplo basta con estudiar el Nearest Neighbors regression de la sección Examples de 1.6.3.

3.4.3 Notas

Aunque el libro no lo comenta dependiendo del problema puede ser necesario escalar las variables para una correcta aplicación del KNN, para los otros procedimientos comentados en este tema no suele ser necesario. Ver en el tema 2 el ejemplo sobre clasificación en que se empleaba el escalamiento.

3.5 Árboles de decisión

Los árboles de decisión (decision trees) son modelos de predicción que permiten la clasificación y la regresión, se basan en grafos con estructura de árboles. Los árboles de decisión son modelos del tipo denominado de "caja blanca", son intuitivos y fáciles de interpretar y entran dentro de la denominada Inteligencia Artificial explicable, por contra los resultados obtenidos de un modelo de "caja negra" hay que admitirlos per se, no son explicables. Estas técnicas tanto para clasificación como para regresión están explicadas en el capítulo 6 de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition y su correspondiente notebook 06_decision_trees.ipynb de handson-ml3-main.

3.5.1 Clasificación

La clasificación usando árboles de decisión está explicada en las siguientes secciones del libro.

En Training and Visualizing a Decision Tree se comenta cómo visualizar los árboles usando los datos de flores iris ya comentados en KNN pero usando las medidas de los pétalos en lugar de la de los sépalos. En Making Predictions se comenta cómo el procedimiento realiza predicciones. El algoritmo es capaz de proporcionar las probabilidades de pertenencia a clases lo que se explica en la sección Estimating Class Probabilities. El algoritmo en el que se basa el procedimiento se comenta en The CART Training Algorithm, Computational Complexity y Gini Impurity or Entropy?. Para terminar cuestiones de selección de parámetros para mejora del entrenamiento se ven en Regularization Hyperparameters. En esta última sección se usa otro generador de muestras aleatorias make_moons. En la sección Instability se comenta limitaciones del procedimiento.

El código del ejemplo se halla en el notebook 06_decision_trees.ipynb de handson-ml2-master. Para el correcto funcionamiento de notebook es necesario instalar el paquete python-graphviz desde Anaconda Navigator. Si se quiere hace uso del comando dot para exportar los árboles es necesario instalar pydot.

La documentación de scikit-learn Decision Trees da una información muy similar a la del libro con la diferencia de que el árbol se muestra con más niveles, solo sería necesario consultar la información de la API de DecisionTreeClassifier y la sección 1.10.5 por sus aspectos prácticos. La sección 1.10.3 Multi-output problems queda fuera de los objetivos de la asignatura, se puede leer como ampliación.

3.5.2 Regresión

La regresión usando árboles de decisión está explicada en las siguientes sección del libro titulada Regression. La documentación de scikit-learn Decision Trees da una información muy similar a la del libro, solo sería necesario consultar la información de la API de DecisionTreeRegressor y la sección 1.10.5 por sus aspectos prácticos.

3.6 Actividades autoevaluables

3.6.1 Naive Bayes

1. ¿Qué parámetros son aprendidos durante el entrenamiento de Naive Bayes?

2. ¿Qué diferencia a cada uno de los procedimientos estudiados?

3. ¿Cuál es uno de los usos más frecuentes de Multinomial Naive Bayes y porqué?

4. Ventajas y desventajas

3.6.2 KNN

1. ¿Qué parámetros son aprendidos durante el entrenamiento de KNN?

2. Describir el procedimiento KNN como se implementa en Implementing Knn in Python.

3. ¿Cómo se deshacen empates? Realizar una búsqueda por Internet.

4. ¿Es la distancia euclídea la única posible en scikit-learn para ordenar los vecinos?

3.6.3 Árboles de decisión

Realizar los ejercicios que se encuentran en la sección Exercices al final del capítulo 6 de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. El último ejercicio constituye una implementación del procedimiento Random Forest que se ve con más detalle en la asignatura Aprendizaje Automático 2.

3.7 Actividades evaluables

La actividad evaluable asociada a este tema y al módulo 1 de la asignatura va a constituir en un ejercicio de clasificación empleando los tres métodos descritos en el tema y la aplicación de varias de las técnicas estudiadas en el tema 2. Para ello habrá que crear un notebook con celdas de tipo code y de tipo markdwon, en las de este último tipo se incluirán comentarios de lo que se esté haciendo en las de tipo code o sobre los resultados.

La fecha de entrega de la actividad aparece en la plataforma.

3.7.1 Datos

Los datos provienen de la web InsideAirBnB, dedicada al estudio de los alquileres vacacionales ofrecidos en la plataforma AirBnB. Es necesario descargar el fichero airbnb.csv de la carpeta datos de Alf. Este fichero es una versión editada, a fin de facilitar la tarea, del listado original de información sobre las ofertas existentes, para la ciudad de Madrid, en abril de 2017. Contiene 13321 registros con 11 campos cada uno, correspondientes a diferentes características de cada oferta de alojamiento.

El fichero se puede descargar a mano y colocarlo en un directorio desde donde lo cargue el notebook.

La tarea de clasificación consistirá en clasificar los datos según el tipo de alojamiento, definido en el campo room_type, a partir del resto de características. Es decir, en room_type estarán codificadas las clases y en el resto de campos los atributos.

3.7.2 Estudio estadístico y limpieza de datos

Se realizará un breve estudio estadístico de los datos numéricos y de la variable room_type se contarán los valores de cada clase. Si las clases no estuvieran balanceadas habrá que usar los mecanismos que puedan tener los algoritmos de clasificación para tratar con este

caso. Para Naive Bayes ya se habla de esto en la actividades autoevaluables, para los otros métodos consultar en la documentación de sus APIs los parámetros de los constructores para ver si hay mecanismos para balancear.

En cuanto a la limpieza de datos estudiar si hay datos faltantes, transformar datos categóricos y escalar datos numéricos haciendo uso de pipelines cuando sea posible.

3.7.3 Entrenamiento y validación

1. Dividir los datos en conjunto de entrenamiento y test de manera que el conjunto de test sea un 20% del total.

2. Evaluar los modelos midiendo la exactitud (accuracy) usando validación cruzada para los tres métodos estudiados utilizando los parámetros por defecto de los tres métodos, salvo lo comentado para el balance, para cross_val_score usar cv=10. Comparar los resultados de los tres modelos.

3. Realizar la afinación de hiperparámetros para KNN y Árboles de decisión empleando GridSearchCV, Naive Bayes no tiene hiperparámetros que afinar.

   (a) Para KNN buscando el valor óptimo de K.

   (b) Para Árboles de decisión: variando max_leaf_nodes entre 2 y 50 ambos inclusive[1], min_samples_split entre 2 y 6 ambos inclusive y max_depth entre 1 y 20 ambos inclusive. Ver la documentación de la API.

3.7.4 Valoración

Se valorará la creación del código que realice los requisitos enumerados, la presencia de comentarios de lo que hace el código y sus resultados y las comparaciones entre los resultados obtenidos por los distintos algoritmos, así como algún gráfico que muestre los resultados. Para terminar incluir un apartado de conclusiones. Tanto los comentarios como las conclusiones no es necesario que sea extensos, sino que describan de forma concisa.

[1] range(a, b) incluye a pero llega hasta b-1

3.8 Soluciones actividades autoevaluables

3.8.1 Naive Bayes

1. Usando la notación de $P(y \mid x_1, \ldots, x_n) \propto P(y) \prod_{i=1}^{n} P(x_i \mid y)$, los parámetros aprender son: las probabilidades de cada clase/etiqueta $P(y)$ y las condicionadas de cada característica por cada clase $P(x_i \mid y)$.

2. La principal diferencia entre los 4 procedimientos estudiados está en la distribución que se supone para las probabilidades condicionadas de cada característica por cada clase $P(x_i \mid y)$. En Gaussian Naive Bayes la distribución se supone gaussiana. En Multinomial Naive Bayes es multinomial. En Complement Naive Bayes la distribución es también multinomial pero las muestras no están balanceadas, no tienen aproximadamente el mismo número de elementos. En Bernoulli Naive Bayes la distribución es la de Bernoulli.

3. Multinomial Naive Bayes se usa frecuentemente en la clasificación de textos. La distribución multinomial es apropiado para características que representan recuentos o tasas de recuento y dado que la vectorización de textos está relacionada con el recuento de palabras o su frecuencia de aparición, Multinomial Naive Bayes se adecua a la clasificación de textos.

4. Ventajas y desventajas:

   (a) Ventajas

      i. Es extremadamente rápido tanto para el entrenamiento como para la predicción.
      ii. Proporciona predicciones probabilísticas sencillas.
      iii. A menudo es muy fácil de interpretar.
      iv. Tiene muy pocos (si alguno) parámetros ajustables.

   (b) Desventajas

      i. La suposición sobre los datos es muy estricta y esto hace que pueda ser peor que otros clasificadores más complejos.

3.8.2 KNN

1. En KNN no hay parámetros que se aprendan durante el entrenamiento, el método simplemente clasifica una determinada instancia en función de qué datos etiqueta-

dos están más cerca de la instancia.

2. La clasificación de los elementos del conjunto de test se realiza así:

   (a) Se calcula para cada dato del conjunto de test la distancia a cada dato del conjunto de entrenamiento.

   (b) Se ordenan los datos del conjunto de entrenamiento en función de las distancias calculadas de menor a mayor.

   (c) Se eligen los K primeros, o sea, los K vecinos más cercanos.

   (d) Se asocia al elemento del conjunto de test la clase predominante en los K vecinos más cercanos.

3. Si solo hay dos clases se elige K impar. Si no siempre podría darse el caso de empates, por ejemplo si el número de clases es 3 y K es 5 podría haber 2 vecinos de una clase, 2 de otra y 1 de la restante. En este caso conviene tener una regla heurística: seleccionar la clase que contenga al vecino más próximo, seleccionar la clase con distancia media menor, etc.

4. El parámetro metric de KNeighborsClassifier permite pasar un objeto de clase DistanceMetric que permite es uso de diferentes métricas como la euclídea, la Manhattan, la Chebyshev, etc. Estas métricas también pueden depender del tipo de datos: reales, enteros, booleanos, etc.

3.8.3 Árboles de decisión

En el apéndice A de Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition y en su correspondiente notebook están las soluciones.